



LOGISTICS OPTIMIZATION

An AI-Native Platform for Delivery, Fleet & Warehouse Operations

Synthetic data generation, machine learning & optimization, and an executive dashboard — built end to end as one connected system.

CEOs

COOs

Operations Directors

Supply Chain Managers

Investors

Six acts, nineteen topics

1

Why This Matters

The business problem, and where AI changes the economics

2

The Data Foundation

A causally-consistent synthetic dataset, built to be learned from

3

The Intelligence Layer

Machine learning, deep learning, optimization, explainability

4

The Product

A nine-module executive dashboard, live and interactive

5

Business Impact

Worked case studies and a quantified ROI estimate

6

Getting to Production

Architecture, security, cloud deployment, roadmap



Why This Matters

The business problem, and where AI changes the economics of moving things around.

Logistics runs on thin margins and manual judgment

8–10%

of global GDP is spent on logistics costs

~\$95B/yr

lost in North America alone to traffic, missed time windows and manual planning errors

~20% CAGR

projected growth of AI spend inside supply-chain operations through the early 2030s

Every one of those numbers describes the same underlying gap: decisions — which route, which vehicle, how much stock, which order is at risk — are still made on averages and gut feel, not on the data the business already collects.

Sources: Grand View Research / IMARC / Coherent Market Insights (logistics market sizing); FutureCar (North American inefficiency loss); MarketsandMarkets / Precedence Research (AI-in-supply-chain growth). Estimates vary across providers — shown as an order-of-magnitude range, not a precise forecast.

The same pain points show up inside our own network



11.3% of deliveries run late

against an 88.7% on-time rate across 50,150 orders — and adverse weather or heavy traffic reliably makes it worse (+2–3pp).



77% of routes carry under 40% of vehicle capacity

median load is just 7% — vehicles and drivers are routinely dispatched far below what they could carry.



44% of the fleet is past 8 years old

219 of 500 vehicles, where unplanned-failure rates climb sharply and maintenance cost follows.

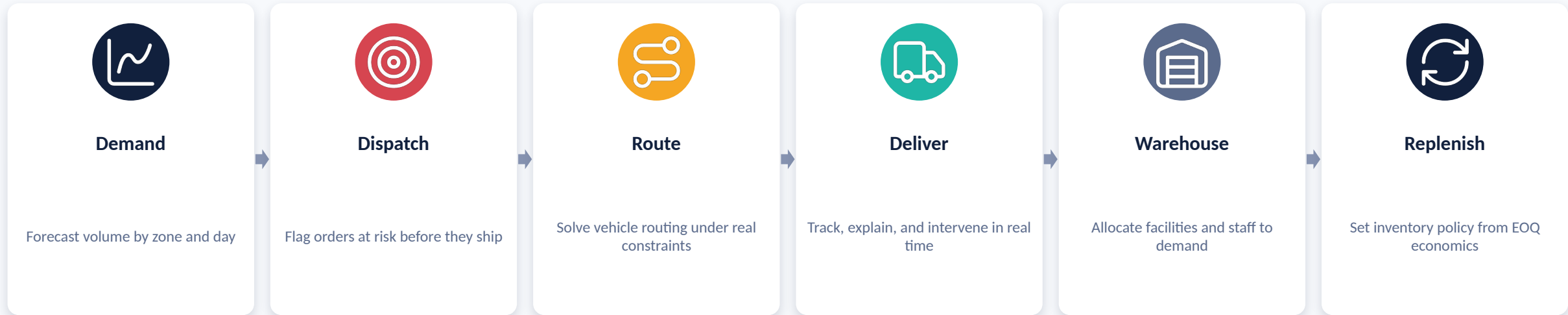


1.36% of warehouse-SKU snapshots are stocked out

enough to block fulfilment at the affected sites during peak windows.

All figures measured on the 50,150-order synthetic dataset generated for this project — see Act 2, The Data Foundation.

Where AI intervenes across the delivery lifecycle



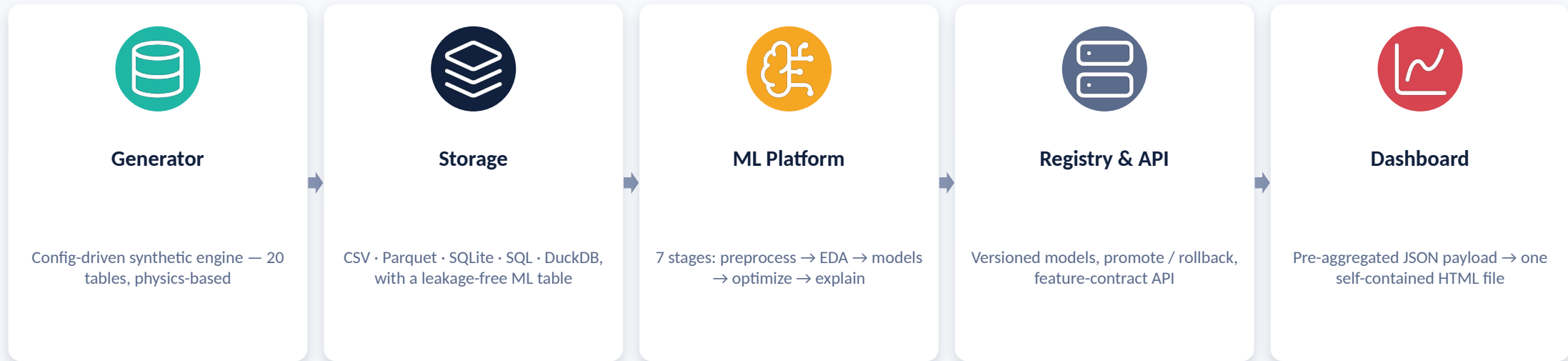
- Each stage of this deck maps to one of these six touchpoints — the dataset makes them measurable, the platform makes them automatable, the dashboard makes them visible.
- None of this replaces dispatcher and planner judgment — it gives them a ranked, explained shortlist instead of a blank screen.



The Data Foundation

A causally-consistent synthetic dataset, engineered so there is something real to learn from it.

One connected system, four repositories



- Every stage reads real output from the stage before it — the dashboard's numbers trace back to the same 50,150-order run the ML platform trained on.
- Each stage degrades gracefully: an optional library that's missing skips its model and says so, rather than aborting the run.

Built to be reproduced, and to scale on command



Same seed, same bytes

Every generator draws from its own named RNG stream, derived from the project seed via a keyed BLAKE2b hash. Reordering or adding a table never perturbs the others.



One number changes the scale

project.scale multiplies every entity count together: 0.05 for a 50,150-order sample (~66s), 1.0 for the full 1M-delivery / ~17M-row spec.



Five export formats, one run

CSV, Parquet, SQLite, portable SQL, and DuckDB all come out of the same run. A failure writing one format never aborts the others.



Validated, not assumed

26 automated checks: referential integrity across 13 foreign keys, physical plausibility, missingness rates on target, and zero leakage columns surviving preprocessing.

Numbers that are built, not drawn

20

related tables, from dimensions to derived ML features

50,150

orders in the sample run used throughout this deck

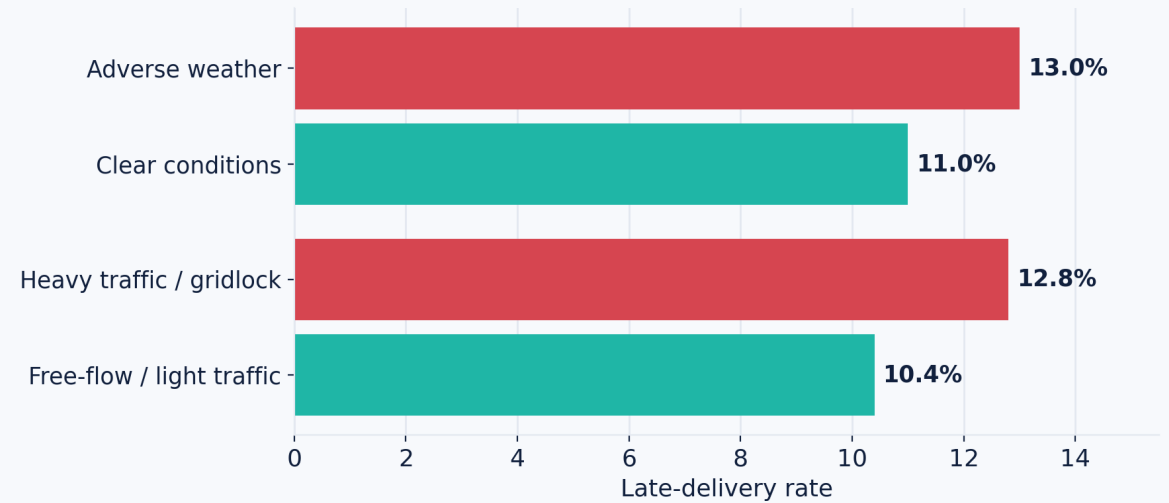
1,000,000

deliveries at the full configured scale (1.0x)

9

business domains: courier, e-commerce, grocery, pharma, manufacturing...

Environment is a genuine upstream cause of delay



```
effective_speed = base_speed ÷ (traffic × weather)
duration = travel_time + service + handling
delay = (pickup_lag + duration) - promised_SLA
```

SLA calibration lands the fleet on the configured 88.5% on-time target (88.7% achieved) by scaling promised windows with a single constant, preserving relative SLA structure across priorities and domains.

Geography and freight physics are enforced, not decorative



Geography is real

Delivery zones only exist in cities that have a warehouse, and each city is served by its nearest site. Median distance 21 km — 18.5 km urban, 27 km suburban, 51 km rural.



Domains behave differently

Freight domains carry heavy loads with a long-haul tail — manufacturing runs a 211 kg median load and a 1,458 km p95 distance. Courier and grocery stay last-mile: 0.8–2.7 kg, ~50 km at p95.



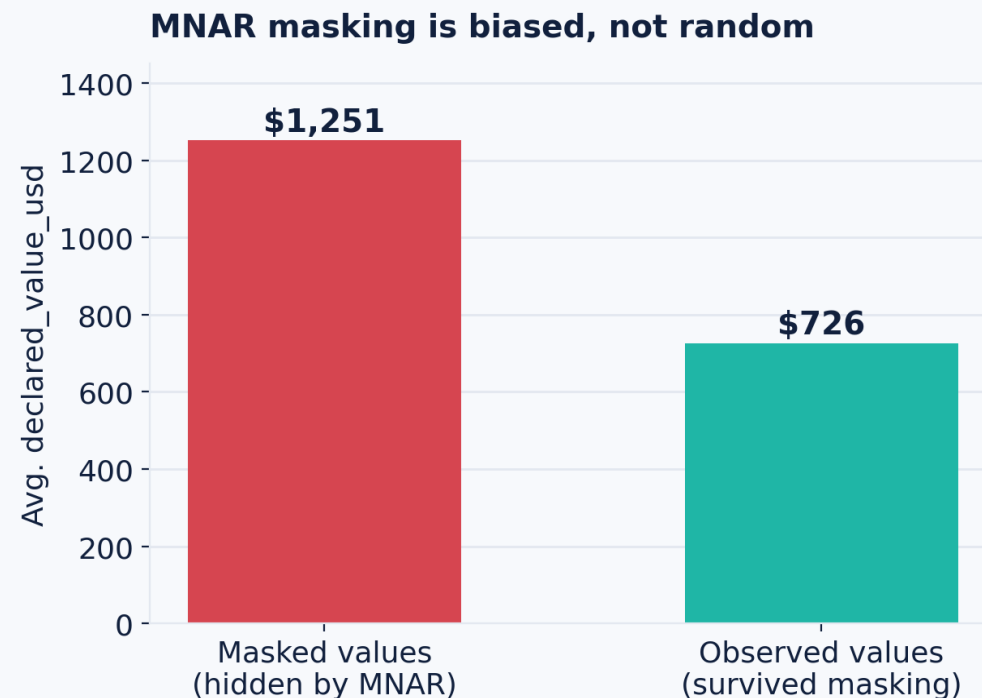
Routes are multi-stop

Orders are clustered by warehouse-day into runs of about 5 stops sharing one vehicle and driver, then rolled up into 9,764 route facts with consolidation savings on distance.

Three missingness mechanisms, deliberately different

Mechanism	Probability depends on	Example rule
MCAR	Nothing	orders.package_weight_kg masked at 5%, independent of any value
MAR	An observed column	orders.actual_delivery_ts masked depending on status
MNAR	The hidden value itself	orders.declared_value_usd — high values are hidden more often

18 rules ship by default. Achieved rates track configured targets to within 0.6 percentage points. Every masked cell is written to `ground_truth/` with its true value, so imputation strategies can be scored, not guessed at.



MNAR bias is real and measurable: naive mean-imputation on `declared_value_usd` would understate true values by roughly 42% — exactly the failure mode this mechanism is built to expose.

15 anomaly classes, every one labelled

Operational disruptions

- Demand spikes & extreme weather events
- Traffic gridlock
- Vehicle breakdowns — delay + cost spike + possible failure
- Driver no-shows
- Warehouse stockouts
- Regional fuel-price shocks
- Address errors
- Fraudulent orders

Data-quality defects

- GPS jumps — position teleports with impossible implied speed
- GPS flatlines
- Sensor drift
- Duplicate orders (-DUP suffix)
- Negative durations
- Order-of-magnitude cost outliers

Every anomaly is tagged in the affected table's `anomaly_flags` column, so detection models have labelled ground truth to train and score against.



The Intelligence Layer

Machine learning, deep learning, optimization and explainability — seven stages, one config-driven platform.

Seven stages, run from one configuration file



- Preprocessing splits temporally (train $\leq$ Oct 6, test from Nov 6) because predicting SLA breaches is forecasting — a random split would let the model read the future.
- Every heavy dependency is optional: XGBoost, LightGBM, CatBoost, OR-Tools, SHAP, LIME, UMAP, PyTorch, FastAPI all degrade gracefully and report their status via ``logisticsml.cli deps``.
- This run: 50,150 orders, 137 engineered features, nine supervised model families benchmarked on one split.

137 features — and a leakage bug we caught ourselves

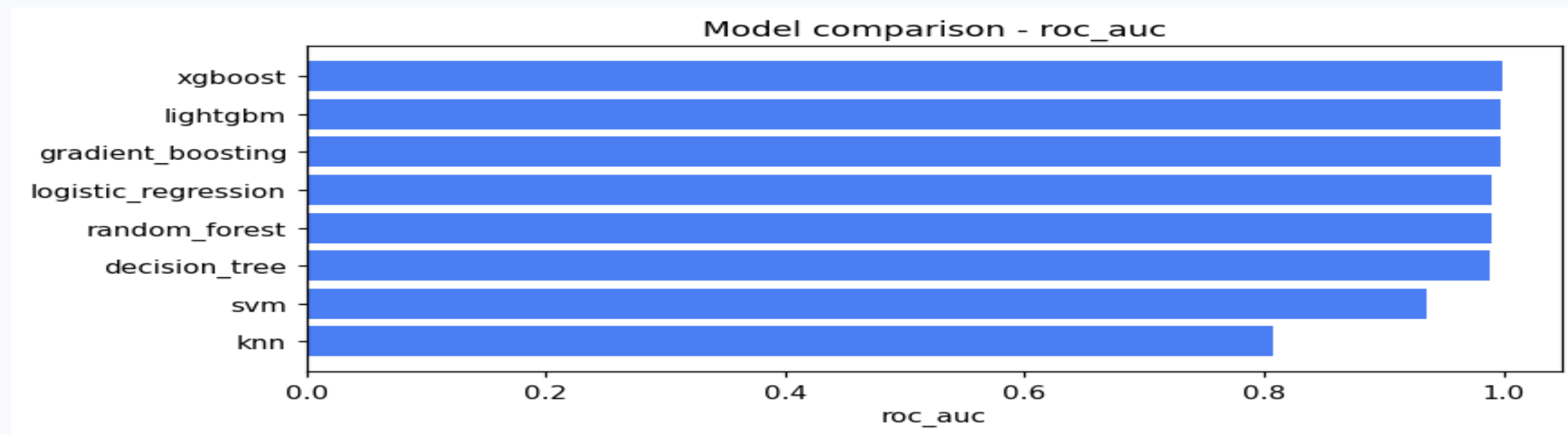
- Temporal features (including cyclical sin/cos encodings), geospatial, package, resource, environmental, rolling-history and efficiency features, denormalised from orders against every dimension table.
- Rolling aggregates are leakage-aware: every historical feature is computed with `.shift(1)`, so a row never sees its own outcome.
- `ml_clean` applies imputation, outlier clipping, ordinal encoding, missing indicators and constant-column dropping — every decision logged to `preprocessing_metadata.json` so the transform can be replayed.



Found in verification, not by the user

- First run: ROC-AUC 1.000. That's leakage, not skill.
- `ml_features` still carried `is_late`, `delay_minutes` and realised costs — the model was scoring its own answer key.
- Fixed with an explicit 25-column leakage guard (`data.leakage_columns`), applied after the target is extracted.
- Honest result: 0.998 — genuinely high, because delay is a deterministic function of distance, traffic, weather and SLA, all of which remain legitimate features. The model's top drivers (`sla_hours`, `lead_time_hours`, `congestion_index`) match the generator's own causal logic.

Nine model families, one held-out split



Model	ROC-AUC	Precision	Recall	F1
XGBoost	0.998	0.960	0.939	0.949
LightGBM	0.998	0.979	0.925	0.951
Gradient Boosting	0.997	0.934	0.921	0.927
Random Forest	0.990	0.878	0.821	0.849

Late-delivery-at-dispatch task, temporal split, 137 features, 50,150 orders. Full nine-model table (incl. SVM, KNN, Decision Tree, Logistic Regression) in the ML platform report.

Two honest framings of the same question



At dispatch

0.998

ROC-AUC

lead_time_hours is known once a vehicle and driver are assigned — the model can use it, and it's the single strongest predictor available.



At order entry (pre-dispatch)

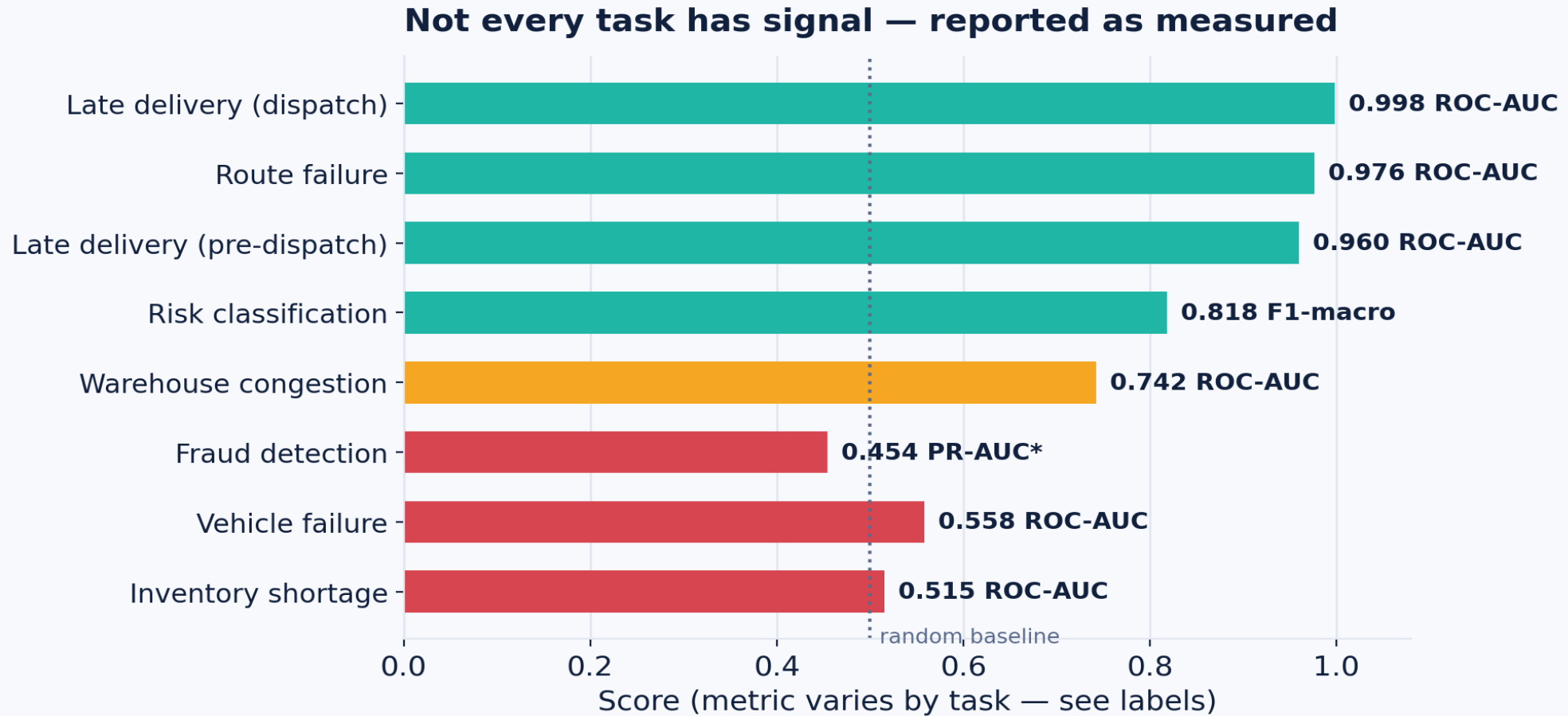
0.960

ROC-AUC

Strips lead_time_hours and everything else known only after assignment. Use this task if you want to quote a delivery promise the moment the order is placed.

The 0.038 gap between the two is the measured value of knowing vehicle/driver assignment before predicting — not a modelling artifact.

Not every task has signal — and the platform says so



*Fraud PR-AUC of 0.454 is 567x the 0.0008 base rate. Inventory shortage is near-random by construction: stockouts are injected independently per snapshot.

Five architectures, native PyTorch, labelled honestly



MLP



Autoencoder



LSTM

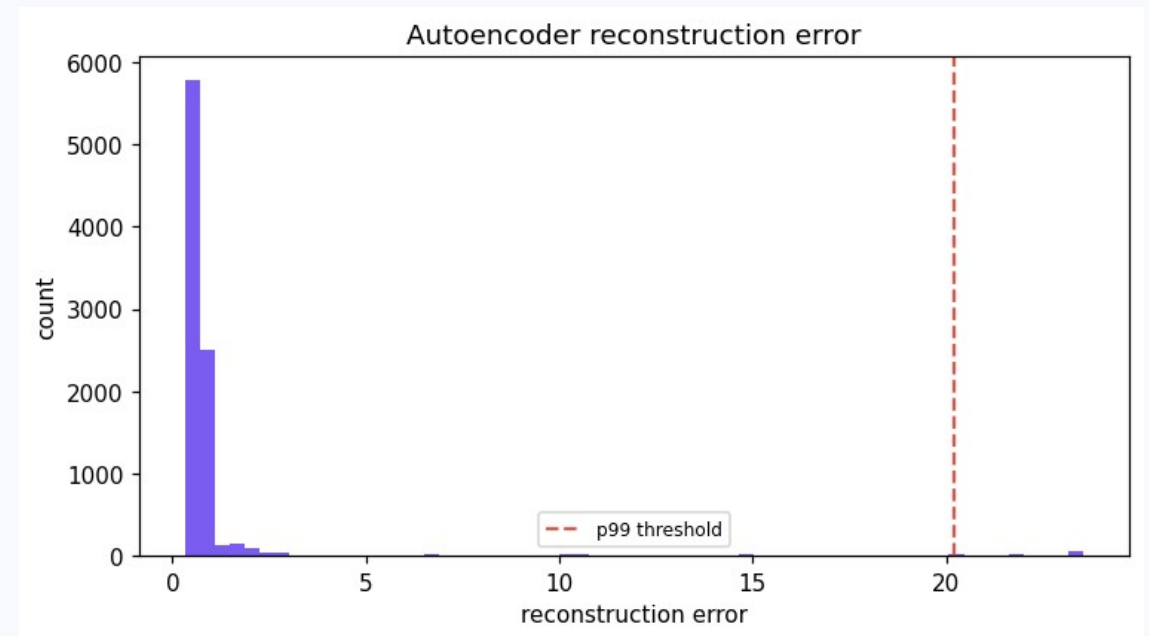


Transformer-style



TabNet (sparsemax)

- All five implemented natively in PyTorch (deep_torch.py) — MLP and autoencoder classics, an LSTM for sequence structure, an FT-Transformer-style model with attention over feature tokens, and TabNet with a from-scratch sparsemax feature-selection layer.
- **This sandbox had no GPU/torch runtime available (network and disk constraints) — every deep-learning result on this slide runs through scikit-learn stand-ins, explicitly labelled APPROXIMATION in every table and report.**
- The full PyTorch path ships ready to run on a machine with torch installed; nothing here is placeholder code.



Eight solvers across four problem families



VRP / CVRP / VRPTW

OR-Tools RoutingModel, with a nearest-neighbour + 2-opt fallback



Driver scheduling

CP-SAT model over shift coverage and labour cost



Fleet allocation

CP-SAT assignment with soft subcontracting



Warehouse allocation

Capacitated facility location, CP-SAT, vs a proper greedy baseline



Inventory (EOQ)

Closed-form economic order quantity with safety stock



Genetic algorithm

From-scratch, benchmarked on the same instance as OR-Tools



Simulated annealing

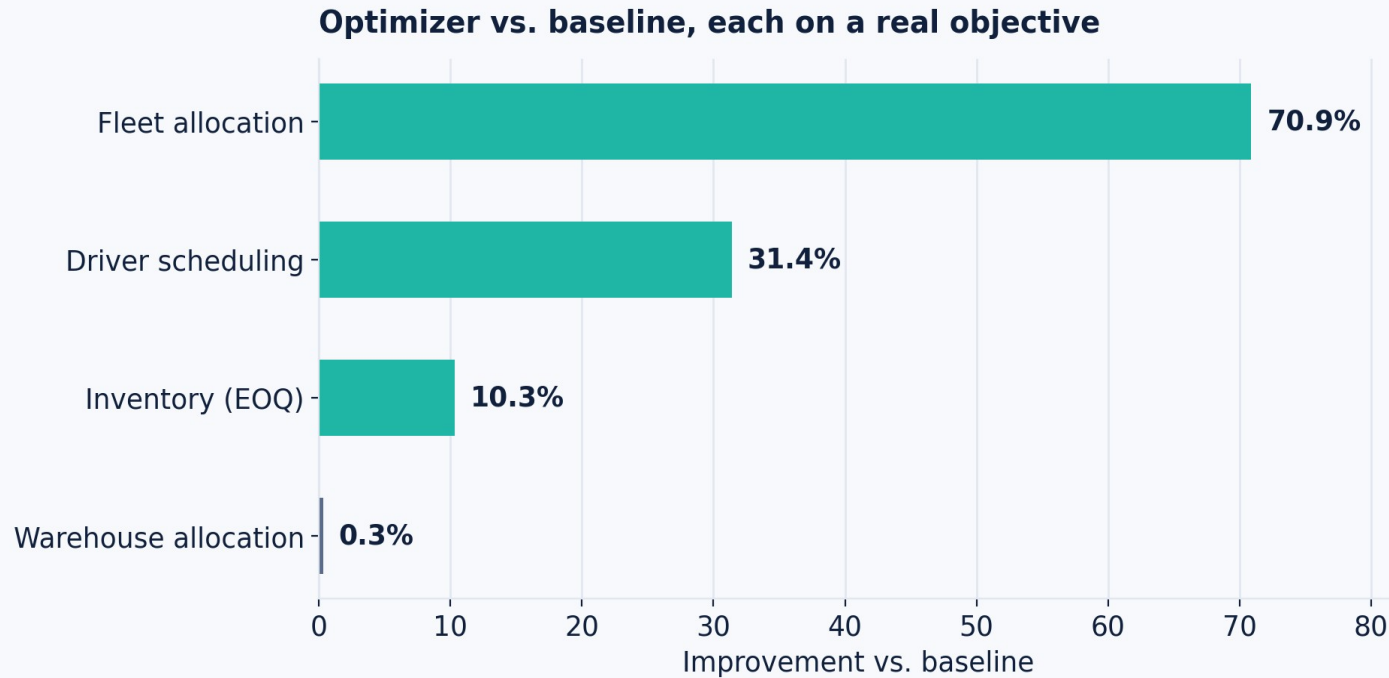
From-scratch — 0.7% off exact, in 1/90th the time



Particle swarm

From-scratch, fastest wall-clock of the four methods

Every result measured against a real baseline

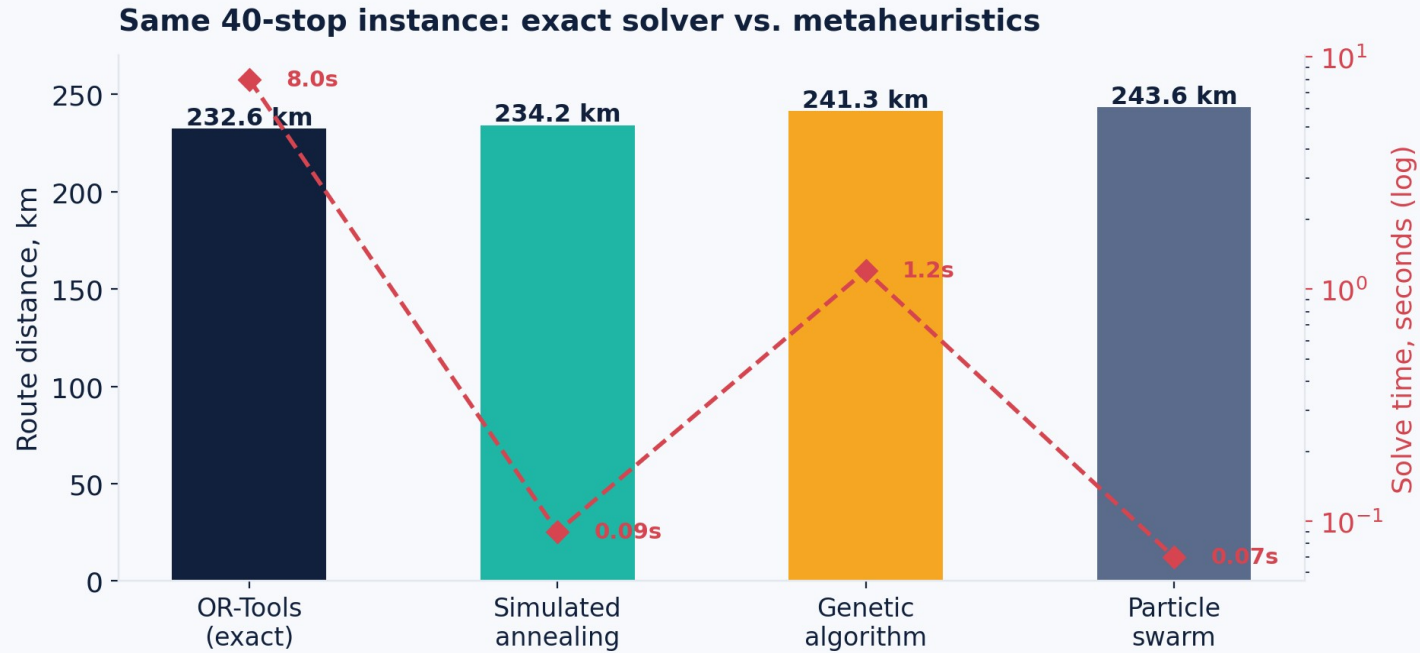


Warehouse allocation's baseline is a proper greedy facility-location heuristic — 0.3% is the honest gap to CP-SAT optimal, not a weak result.

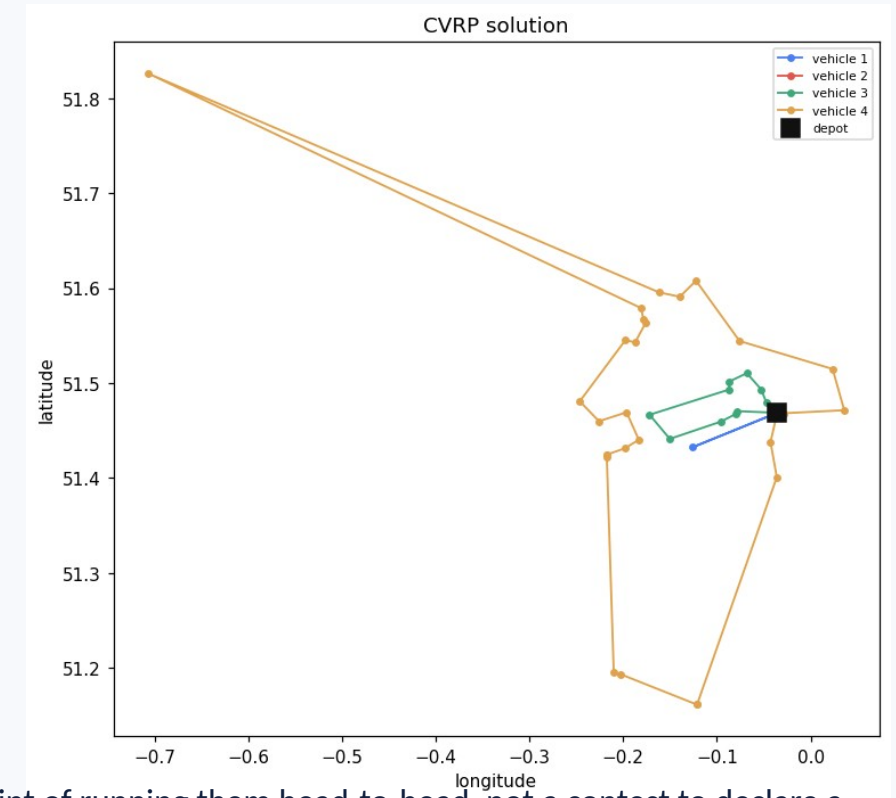
Problem	Objective	Baseline
Driver scheduling	\$6,028	\$8,782
Fleet allocation	\$3,640	\$12,519
Warehouse allocation	48,935	49,073
Inventory (EOQ)	\$9.38M/yr	\$10.45M/yr

Warehouse allocation improves only 0.3% — and that's the honest number. Its baseline is greedy facility location, which is provably near-optimal here. An earlier version compared against an “open every site” baseline and reported a meaningless 66%.

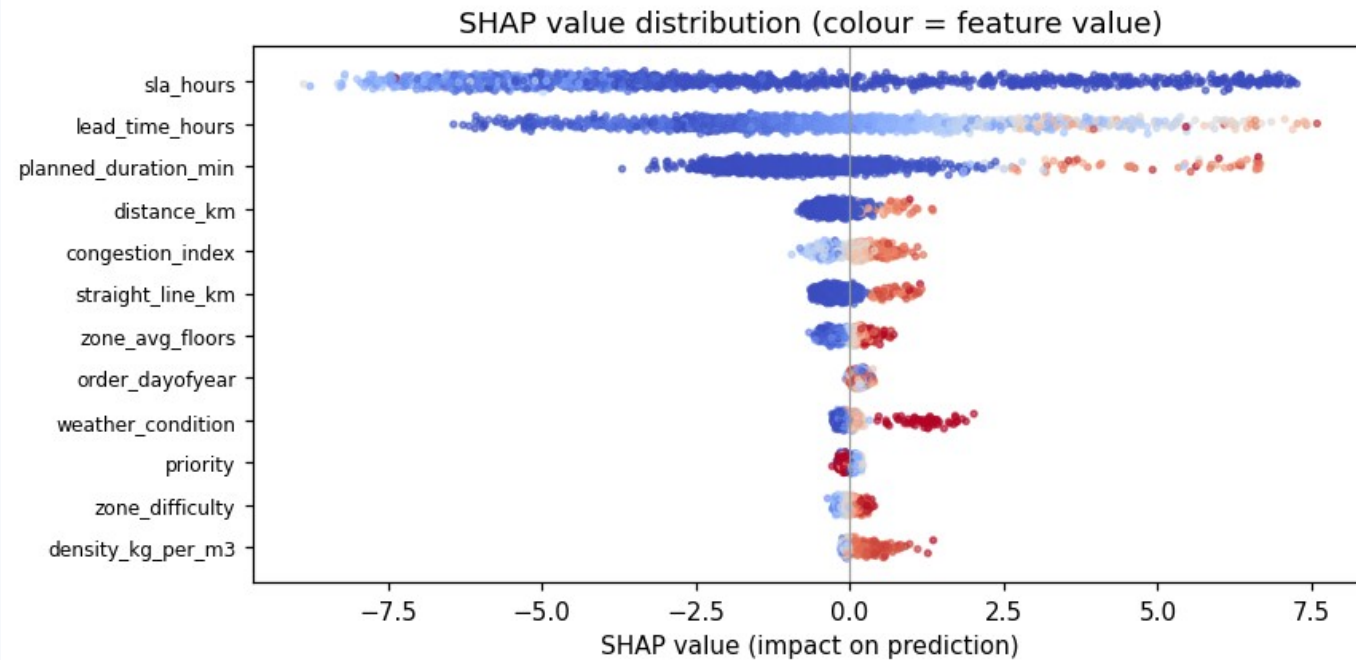
The same 40-stop instance, four ways



Simulated annealing lands within 0.7% of the exact solver in 1/90th of the time — the trade-off is the point of running them head-to-head, not a contest to declare a single winner.



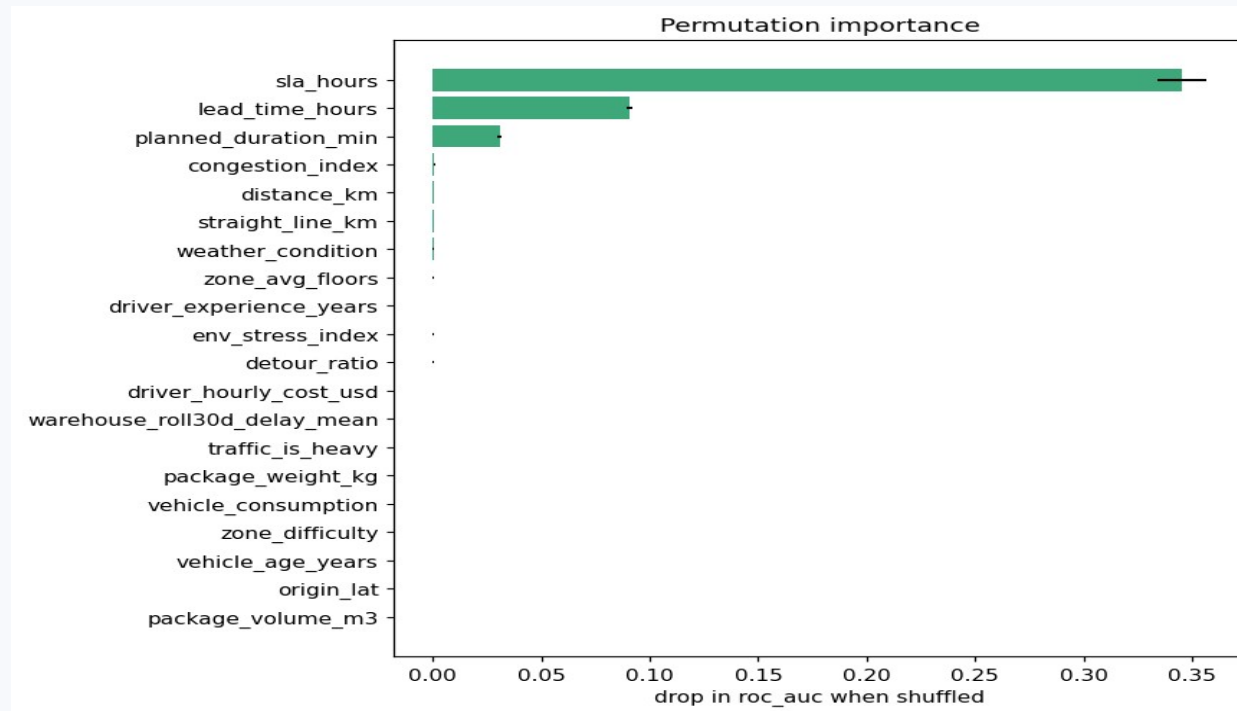
SHAP attribution, computed natively



A packaging bug, not a modelling one

- The released shap package fails against XGBoost 3.x — it can't parse the JSON-encoded base_score.
- The platform calls XGBoost's and LightGBM's native TreeSHAP directly instead — the same exact Shapley values, from the library that owns the tree structure.
- **Top attribution: sla_hours (39%), lead_time_hours (22%), planned_duration_min, congestion_index, distance_km.**

Two methods, cross-checked — and wired into governance



- SHAP vs. permutation importance rank correlation: 0.61 — sharp disagreement would flag correlated features or an unstable model; this level says the two methods broadly agree.
- LIME (in-house weighted-ridge surrogate) and partial dependence available per-instance and per-feature.
- Governance is built in, not bolted on: the model registry versions every run, promotion and rollback are one command, and the prediction API enforces a feature contract — reordering input columns cannot change a prediction. That invariant is covered by a test.



The Product

A nine-module executive dashboard — one self-contained file, live filters, and an honest empty state when data is missing.

Nine modules, one page



Executive Dashboard

Revenue, cost, margin, on-time rate, MoM movement



Operations Center

Demand heatmap, hourly reliability, congestion profile



Fleet Management

Composition, age, energy mix, utilisation, maintenance



Warehouse Analytics

Throughput vs. capacity, automation, stockout exposure



Delivery Monitoring

Interactive map, 7 layers, SLA outcome by priority



Demand Forecast

30-day projection, 95% interval, seasonality decomposition



Optimization Simulator

8 what-if levers, live cost & service recomputation



AI Recommendations

9 ranked, quantified actions, each with its evidence



Explainable AI

SHAP, leaderboard, agreement check, partial dependence

A composite you can trust, not a slide of hopes

LOGISTICS OPTIMIZATION

Executive Dashboard

Region: All

Domain: All

Export

\$9.36M

Revenue

88.7%

On-time rate

42.9%

Margin

\$107

Cost / order

9,764

Active routes

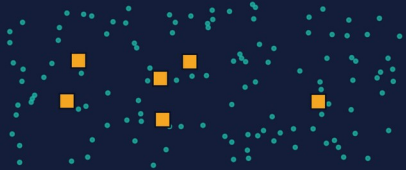
Revenue trend



Fleet composition (500 vehicles)



Delivery Monitoring — live map



AI Recommendations

Adopt the optimised inventory opti...
\$1,076k / yr

Consolidate routes running under 4...
\$497k / yr

Plan replacement for 219 vehicles ...
\$394k / yr

Raise safety stock on the SKUs dri...
\$160k / yr

Explainable AI — SHAP attribution



Illustrative composite of dashboard.html rendered from real payload figures — see the live file for the interactive version.

- **Filters actually filter.**
- A 432-row region × domain × month cube lets the browser genuinely re-aggregate the executive figures — not just swap a label.
- **The simulator is calibrated, not invented.**
- Congestion and weather elasticities are measured late-rate gaps from this dataset; vehicles and drivers are modelled Cobb-Douglas (85/15) toward whichever is scarcer.
- **Forecast accuracy is reported out-of-sample.**
- 16.97% MAPE held out, vs. 7.04% in-sample — the flattering number is not the one we lead with.
- **43 / 43 validation checks**
- in a headless harness: every KPI reconciles to the payload, every chart container renders or shows an honest empty state.

dashboard.html is one self-contained file (0.72 MB) — opens by double-click, no server. This image is an illustrative composite built from the same payload, for a slide that has to work without a browser.



Business Impact

Three worked examples from this dataset, and a quantified — explicitly illustrative — ROI estimate.

Predicting, explaining, and pricing for late deliveries



Predict

The dispatch-time model flags at-risk orders at ROC-AUC 0.998, using features known the moment a vehicle and driver are assigned.



Explain

SHAP shows sla_hours alone accounts for 39% of attribution — the model leans on the same lever a planner would.



Confirm

Adverse weather deliveries run 13.0% late vs. 11.0% in clear conditions — a measured +2.0pp across 7,236 deliveries, not a modelling artifact.



Act

Recommendation: extend promised windows automatically once severity crosses a threshold — quantified impact \$27,854/yr at this sample's scale.

A fleet that's under-loaded, and a stock policy already priced



Fleet consolidation

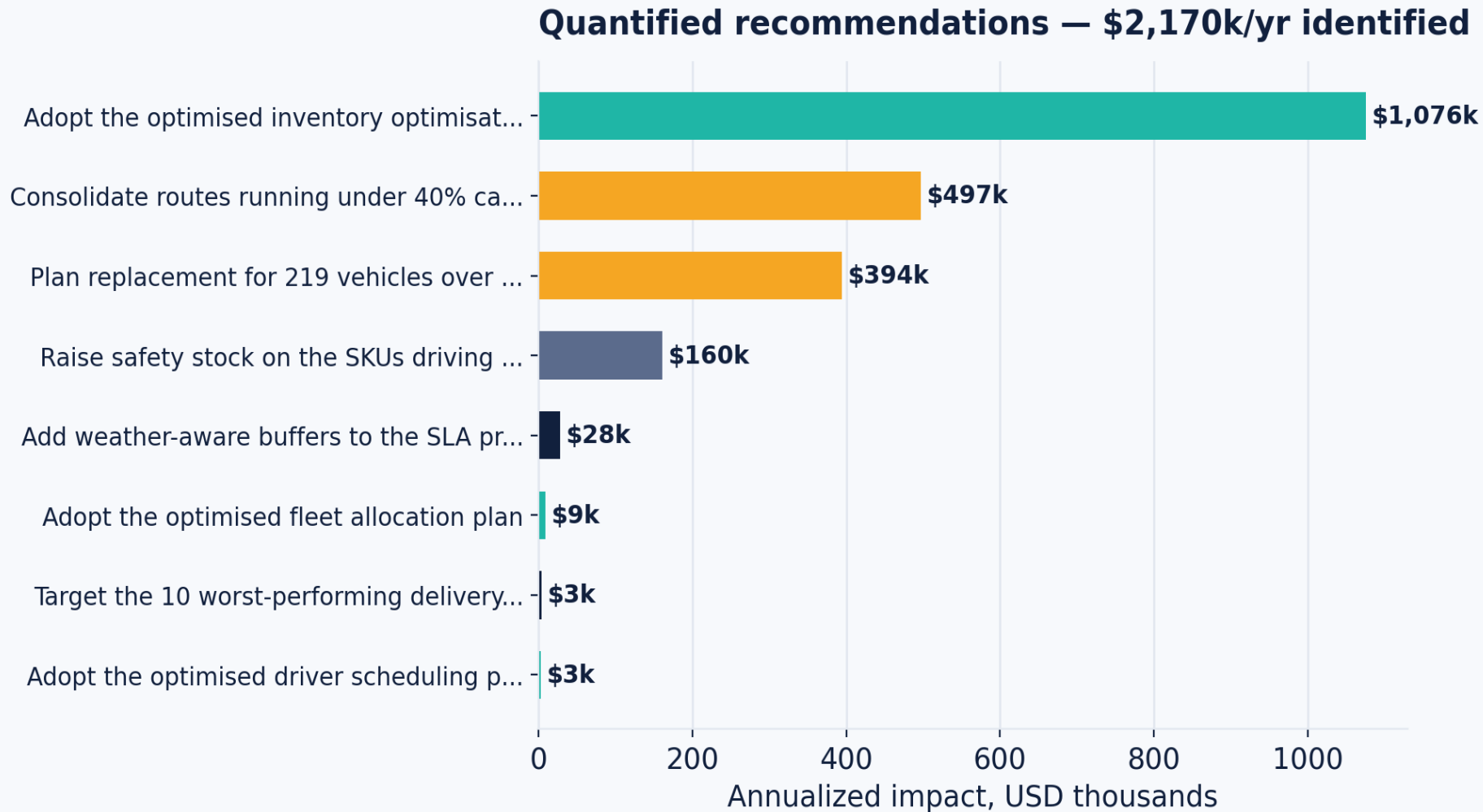
- 77% of routes run under 40% of vehicle capacity; median load is just 7%.
- Recommended: raise the minimum load threshold before dispatch — \$496,688/yr identified at this sample's scale.
- Separately, the fleet-allocation CP-SAT solver already beats a real assignment baseline by 70.9% on its own cost objective — the two levers are complementary.



Inventory policy

- EOQ with safety stock reaches \$9.38M/yr against a \$10.45M/yr baseline — 10.3%, or \$1.076M/yr.
- The single largest lever identified anywhere in this platform, and it's closed-form: no solver required, just the right formula applied consistently.
- Paired with a targeted safety-stock raise on the SKUs driving the 1.36% stockout rate (\$160,274/yr).

\$2.17M/yr in quantified opportunity, at this sample's scale



\$9.36M

Annual revenue

\$5.35M

Annual cost

42.9%

Margin

\$2.17M

Identified opportunity

Read every number on the last two slides as a method demonstration



These figures are illustrative estimates computed on the generated synthetic sample (50,150 orders). They demonstrate the methodology — how a recommendation is derived and priced — not a guaranteed outcome. Real-world impact depends on data quality, integration depth, organizational adoption, and how closely this dataset's physics match your actual operations.

- Data quality: production data is messier than a calibrated generator — expect the first pass to find data problems before it finds savings.
- Integration depth: the impact estimates assume recommendations are actually operationalised into planning, not just reported.
- Scale: these dollar figures scale with order volume; a network 10x this size should expect roughly 10x the opportunity, not 10x the confidence.
- Change management: the largest single lever (EOQ inventory, \$1.076M/yr) requires zero new technology — only the discipline to run the existing formula consistently.



Getting to Production

Architecture, security, cloud deployment, and an honest roadmap for the gap between this build and a production rollout.

Four layers, entirely open-source



Data layer

pandas, NumPy, PyArrow — Parquet, SQLite, DuckDB



Intelligence layer

scikit-learn, XGBoost, LightGBM, CatBoost, PyTorch, OR-Tools



Serving layer

File-backed model registry — versioned, promotable, rollback-able; FastAPI or stdlib HTTP



Presentation layer

Single-file HTML, Tailwind, Plotly, Leaflet — no server to run

Every optional dependency degrades gracefully rather than aborting the run — a design property, not just a library list.

Governance is real today; production hardening is the next step

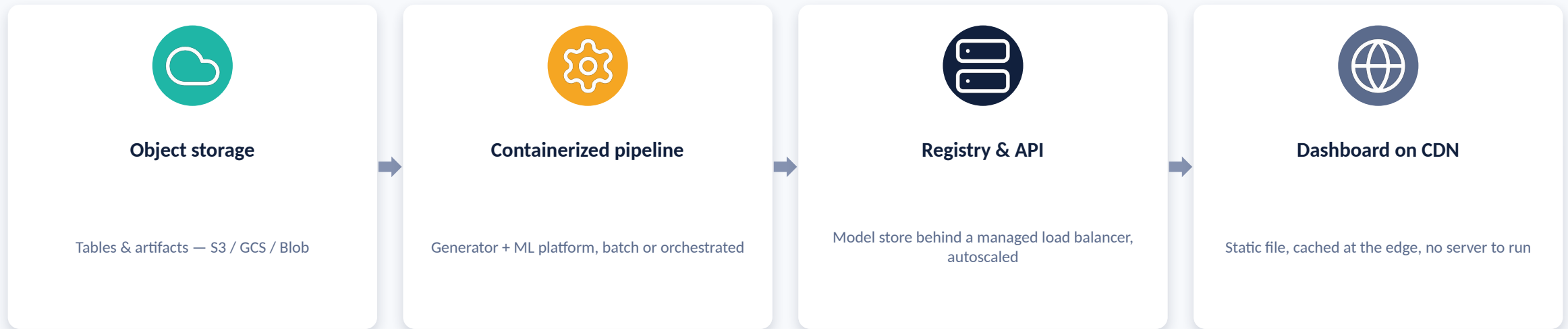
Already built

- Versioned model registry — every run stores estimator, preprocessor, exact feature order and provenance.
- Explicit promotion and one-command rollback; the production version is never pruned.
- Prediction API enforces a feature contract — reordering input columns cannot change a prediction, and that invariant is covered by a test.

Needed before production

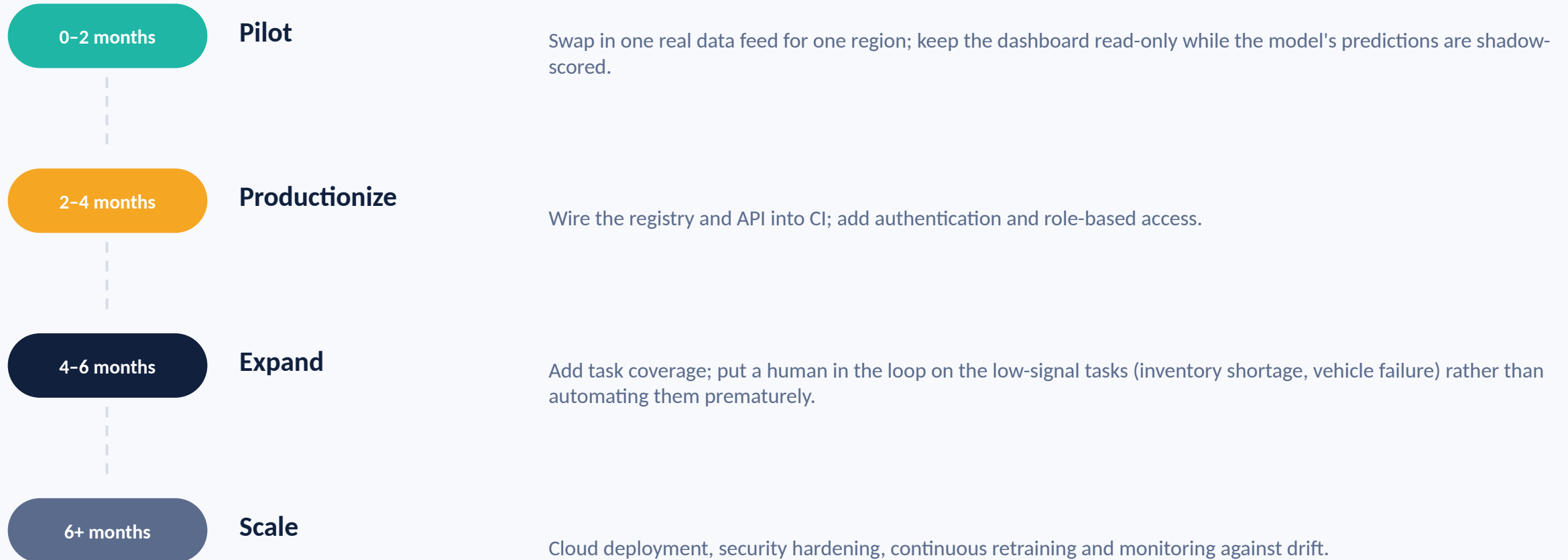
- Encryption at rest and in transit for real (non-synthetic) customer, driver and location data.
- Role-based access control on both the dashboard and the prediction API.
- PII minimisation and masking — this dataset is synthetic and contains none, but a real deployment will.
- Audit logging, and vetting of every optional ML dependency before it reaches production.

A cloud-agnostic reference architecture



- No proprietary service is required to reproduce this deck: the entire pipeline above already runs, identically, inside a local sandbox.
- Swapping in a managed queue, warehouse, or orchestrator (Airflow, Step Functions, Cloud Composer) is additive, not a rewrite.

An illustrative path from this build to production



Illustrative sequencing, not a committed delivery plan — durations depend on data access, integration scope and organizational readiness.



Questions?

Four repositories, one connected system.



logistics_datagen

20 tables · 26 checks passing



logistics_ml

7 stages · 27 checks passing



logistics_dashboard

9 modules · 43 checks passing



This deck

19 topics · every figure sourced

chuvardynskyi@yahoo.com